SG6–HPC – TP4 Méthode du gradient conjugué

Compte Rendu de TP

Binôme :
Cêtre Bastien – Lutz Adrien

TP du : Mercredi 21 janvier 2026

Introduction

Ce travail pratique vise à implémenter la méthode du gradient conjugué pour la résolution de systèmes linéaires, dans un premier temps en version séquentielle, puis en environnement parallèle. La parallélisation repose sur un découpage de la matrice en blocs de lignes et sur l'utilisation de la bibliothèque `mpi4py` pour la gestion des communications MPI. Une attention particulière est portée à l'identification des communications collectives nécessaires et à l'analyse de leur impact sur les performances lors d'exécutions sur plusieurs processus.

1 Implémentation du gradient conjugué parallèle par blocs de lignes

Objectif de la question

L'objectif de cette première question est d'implémenter une version parallèle de la méthode du gradient conjugué par découpage de la matrice en blocs de lignes, en utilisant le modèle de programmation MPI. Il s'agit d'adapter l'algorithme séquentiel vu en cours afin qu'il fonctionne sur plusieurs processus tout en conservant les propriétés de convergence de la méthode.

Rappel théorique

La méthode du gradient conjugué permet de résoudre un système linéaire $Ax = b$, où A est symétrique définie positive. À chaque itération, l'algorithme repose sur :

— un produit matrice-vecteur $v = Aw$,
— plusieurs produits scalaires pour le calcul des coefficients ρ et γ,
— des mises à jour de la solution, du gradient et de la direction de descente.

Dans une version parallèle par blocs de lignes, chaque processus MPI ne manipule qu'un sous-ensemble des lignes de la matrice et des composantes associées des vecteurs. Les produits scalaires nécessitent des réductions globales, tandis que le produit matrice-vecteur requiert la reconstruction du vecteur global.

Principales modifications apportées au code

Le code fourni pour la version parallèle contenait plusieurs sections à compléter. Les modifications majeures réalisées concernent la gestion des communications MPI et l'adaptation des opérations globales de l'algorithme.

Assemblage du vecteur global Chaque processus ne possédant qu'un sous-vecteur local de la direction de descente w, il est nécessaire de reconstruire le vecteur global avant de calculer le produit matrice-vecteur. Cette reconstruction est réalisée à l'aide d'une communication collective `MPI_Allgather`. Concrètement, cette opération est effectuée à l'aide de l'instruction : `MPI.COMM_WORLD.Allgather(w, w_assemb)`, permettant à chaque processus d'obtenir une copie complète du vecteur w avant le calcul local du produit `v = A.dot(w_assemb)`.

Calcul des produits scalaires Les produits scalaires nécessaires au calcul des coefficients ρ et γ sont d'abord calculés localement sur chaque processus, par exemple à l'aide de `s1 = numpy.vdot(g, w)`. Les contributions locales sont ensuite sommées globalement grâce à une réduction MPI : `MPI.COMM_WORLD.Allreduce(s1_loc, s1_assemb, op=MPI.SUM)`. Cette stratégie est utilisée pour l'ensemble des produits scalaires globaux de l'algorithme.

Mise à jour locale des vecteurs Les mises à jour de la solution, du gradient et de la direction de descente sont effectuées localement sur chaque processus, sans communication supplémentaire. Par exemple, la solution est mise à jour via l'instruction `x = x + rho * w`, et le gradient via `g = g + rho * v`. Ces opérations correspondent à des combinaisons linéaires simples et sont naturellement parallélisables.

Critère de convergence Le critère d'arrêt repose sur la norme globale du gradient. Celle-ci est obtenue en calculant localement le produit scalaire `numpy.vdot(g, g)`, puis en effectuant une réduction globale à l'aide de `MPI.COMM_WORLD.Allreduce(g_norm2_loc, g_norm2_glob, op=MPI.SUM)`. La norme relative du résidu est ensuite calculée et comparée au seuil de convergence fixé.

Validation de l'implémentation

La solution globale reconstruite est ensuite comparée à une solution de référence à l'aide de la procédure de correction automatique fournie. Cette étape consiste à vérifier que l'erreur relative entre la solution numérique et la solution exacte reste inférieure à une tolérance prescrite. Cette vérification est réalisée par l'appel à la fonction `automatic_correction_is_solution_correct(n, x_cgp_assemb, epsilon=1.e-3)`, permettant de valider la convergence et la justesse de l'implémentation parallèle.

```
sg6hpc1_14@kyle07:~/Enonce-TD4-improved-py$ mpiexec --oversubscribe -n 4 python3 exercice5_conjugate_gradient_parallel_questmpie
xec --oversubscribe -n 4 python3 exercice5_conjugate_gradient_parallel_question.py
```

```
Ellapsed time to compute iterative (parallel) solution =  0.019014 (s)
(Parallel) computing speed =  37.87 (Kvar/s)
Automatic correction: Congratulation, you pass.
```

FIGURE 1 – Validation de l'implémentation

2 Génération distribuée des sous-matrices et optimisation mémoire

Objectif de la question

L'objectif de cette seconde question est d'améliorer la scalabilité mémoire du code parallèle. La version initiale générait la matrice globale complète sur chaque processus avant d'en extraire un bloc de lignes local, ce qui entraînait une duplication inutile de la mémoire et limitait fortement la taille des problèmes traitables. L'objectif est donc de mettre en œuvre une génération entièrement distribuée des sous-matrices locales, sans construction préalable de la matrice globale.

Rappel théorique

Dans un contexte de calcul haute performance, un algorithme est dit scalable en mémoire lorsque la consommation mémoire par processus reste bornée lorsque la taille du problème augmente. Pour une décomposition de domaine par blocs de lignes, chaque processus doit uniquement stocker les lignes de la matrice et les composantes des vecteurs qui lui sont associées. Lorsque la structure de la matrice est connue à l'avance, il est possible de reconstruire directement les coefficients locaux sans passer par une matrice globale.

Modification de la bibliothèque d'assemblage

La fonction `assemble_block_parallel` de la bibliothèque `exercice5_package.py` a été entièrement réécrite afin de supprimer toute dépendance à la génération de la matrice globale.

Détermination des bornes locales Chaque processus calcule les indices globaux de début et de fin des lignes qu'il doit traiter. Ces bornes sont définies à l'aide des instructions `ibegin = indBlock * (dim / nbBlocks)` et `iend = (indBlock + 1) * (dim / nbBlocks)`, ce qui permet d'identifier précisément le sous-domaine associé à chaque processus.

Allocation mémoire optimisée Contrairement à la version initiale, seule la sous-matrice locale est allouée en mémoire. Cette allocation est réalisée via l'instruction `A_local = numpy.zeros((rows_local, dim))`, où `rows_local = iend - ibegin`. Ainsi, la mémoire utilisée par chaque processus est divisée par le nombre total de processus MPI.

Génération directe des coefficients Les coefficients de la matrice sont ensuite générés directement à partir de leur expression mathématique, sans extraction depuis une matrice globale. La diagonale principale et les termes hors-diagonale sont construits à l'aide des instructions `A_local[i_loc, i_glob] = 4.0`, `A_local[i_loc, i_glob + 1] = -1.0` et `A_local[i_loc, i_glob - 1] = -1.0`, en respectant la structure tridiagonale du problème.

Gestion des conditions aux limites Une attention particulière a été portée à la gestion des conditions aux limites afin de garantir la compatibilité avec la solution de référence. La première ligne globale est traitée spécifiquement via l'instruction `A_local[i_loc, 0] = 1.0`, correspondant à une condition de Dirichlet. De plus, la colonne globale 0 est annulée pour l'ensemble des processus, ce qui assure une reproduction exacte du problème séquentiel.

Modification du programme principal

Afin d'exploiter cette nouvelle fonction d'assemblage, le programme principal a été modifié pour remplacer l'appel à la fonction initiale `assemble_block` par un appel à `assemble_block_parallel(n, size, rank)`. Cette modification garantit que la matrice globale n'est jamais construite et que l'ensemble du calcul repose sur une décomposition mémoire distribuée.

Validation de l'approche

L'exécution du programme sur plusieurs processus MPI montre que l'algorithme du gradient conjugué converge correctement avec cette nouvelle stratégie d'assemblage. La procédure de correction automatique valide numériquement la solution obtenue, confirmant que l'optimisation mémoire n'altère ni la convergence ni la précision du solveur.

```
Host shared a Live Share terminal. Enter to type.mpiexec --oversubscribe -n 4 python3 exercice5_conjugate_gradient_parallel_question
.py --s 4000                    mpiexec --oversubscribe -n 4 python3 exercice5_conjugate_gradient_parallel_question.py --s 4000

Value of iterative (parallel) solution = [1.         0.3660254  0.46410162 ... 0.49038106 0.46410162 0.3660254 ]
Ellapsed time to compute iterative (parallel) solution =  0.085875 (s)
(Parallel) computing speed =  93.16 (Kvar/s)
```

FIGURE 2 – Validation de l'implémentation

3 Analyse des performances et passage à l'échelle faible

Objectif de la question

L'objectif de cette question est d'évaluer les performances de l'algorithme parallèle optimisé en mémoire (Question 2) à l'aide d'un test de passage à l'échelle faible (*weak scaling*). Ce test consiste à augmenter la taille globale du problème proportionnellement au nombre de processus MPI, de manière à conserver une charge de calcul constante par processus, afin d'identifier les limites de scalabilité de l'algorithme.

Rappel théorique

Dans un scénario de *weak scaling*, chaque processus traite une quantité de données constante. Théoriquement, le temps d'exécution total devrait rester constant lorsque le nombre de processus augmente, tandis que la puissance de calcul globale devrait croître linéairement. Toute déviation par rapport à ce comportement idéal est généralement due aux communications collectives et aux limitations matérielles.

Préparation du code pour les mesures

Avant de réaliser les mesures de performance sur des matrices de grande taille, la procédure de correction automatique a été désactivée dans le programme principal. Cette modification correspond au commentaire de l'appel à la fonction `automatic_correction_is_solution_correct`. En effet, cette fonction reconstruit une matrice séquentielle globale sur le processus de rang 0, ce qui impliquerait une allocation mémoire de taille $\mathcal{O}(n^2)$ et rendrait impossible l'étude des performances pour de grandes valeurs de n.

Protocole expérimental

Les tests ont été réalisés en conservant une charge de calcul constante de 1000 lignes par processus. La taille globale de la matrice est donc proportionnelle au nombre de processus MPI. Les exécutions ont été lancées avec l'option `-oversubscribe`, afin de simuler un grand nombre de processus sur un même nœud de calcul.

Les temps d'exécution mesurés ont été exploités à l'aide d'un script Python basé sur `matplotlib`, permettant de calculer le débit global (en Kvar/s) et l'efficacité du passage à l'échelle.

TABLE 1 – Mesures de performances en passage à l'échelle faible

Nombre de processus (p)	Taille globale (n)	Charge par proc	Temps (s)
4	4 000	1 000	0.0526
8	8 000	1 000	0.0859
16	16 000	1 000	0.1816
32	32 000	1 000	0.5041

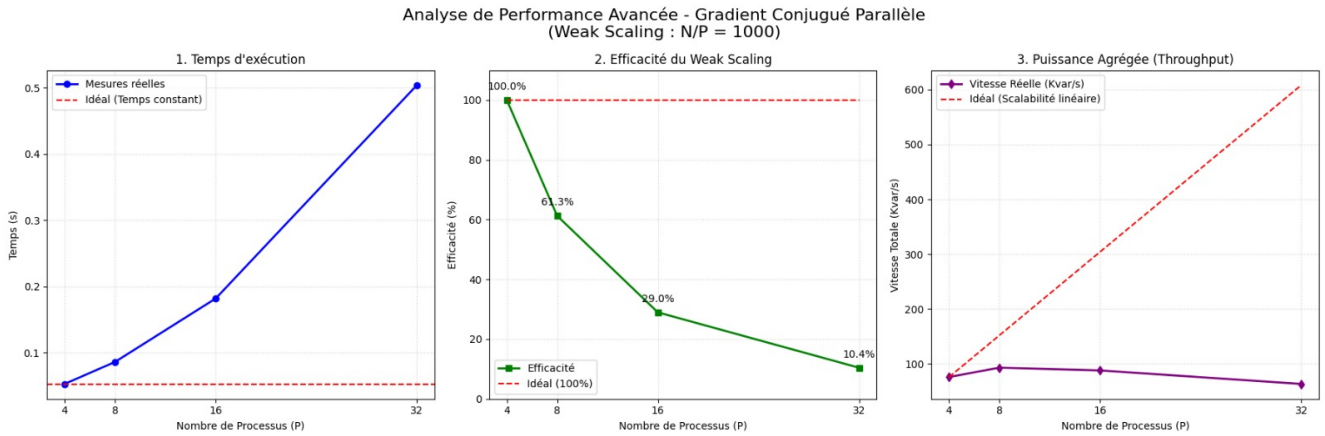


FIGURE 3 – Analyse des performances en passage à l'échelle faible : temps d'exécution, efficacité et débit global

Analyse des résultats et lien avec l'implémentation

La première courbe montre que le temps d'exécution augmente significativement avec le nombre de processus, alors qu'il devrait rester constant dans un scénario de passage à l'échelle faible. Ce comportement s'explique directement par l'implémentation du produit matrice-vecteur, qui repose sur l'appel à la communication collective `MPI.COMM_WORLD.Allgather(w, w_assemb)` à chaque itération du gradient conjugué. Lorsque le nombre de processus augmente, le volume de données échangées croît linéairement avec la taille globale du vecteur, ce qui induit un surcoût de communication dominant.

La deuxième courbe met en évidence une chute rapide de l'efficacité du passage à l'échelle. Cette perte d'efficacité traduit le fait qu'une part croissante du temps d'exécution est consacrée aux synchronisations globales, notamment aux appels répétés à `MPI.COMM_WORLD.Allreduce` utilisés pour le calcul des produits scalaires globaux nécessaires aux coefficients ρ et γ.

Enfin, l'analyse du débit global révèle l'existence d'un point de saturation. La puissance de calcul augmente jusqu'à un nombre modéré de processus, puis décroît lorsque davantage de cœurs sont utilisés. Ce phénomène correspond à un régime de *negative scaling*, où l'ajout de ressources de calcul devient contre-productif. Il s'explique par la saturation de la bande passante mémoire du nœud et par la contention induite par les communications collectives sur des vecteurs de grande taille.

Conclusion

Ce travail pratique a permis de mettre en œuvre et d'analyser en profondeur la méthode du gradient conjugué dans un contexte de calcul haute performance. Après une implémentation parallèle fonctionnelle reposant sur un découpage de la matrice en blocs de lignes, une optimisation mémoire essentielle a été réalisée afin de lever les limitations liées à la duplication de la matrice globale. Cette optimisation s'est révélée indispensable pour traiter des problèmes de grande taille tout en conservant la validité numérique de la solution.

L'étude de performances en passage à l'échelle faible met toutefois en évidence les limites de la parallélisation adoptée. Bien que l'optimisation mémoire permette d'exécuter le code à grande échelle, elle ne garantit pas une bonne scalabilité en temps. À mesure que le nombre de processus augmente, les communications collectives deviennent prépondérantes. En particulier, les appels répétés à `MPI_Allgather` pour la reconstruction des vecteurs globaux et à `MPI_Allreduce` pour le calcul des produits scalaires constituent le facteur limitant principal des performances.

Ces résultats soulignent un compromis classique en calcul parallèle entre simplicité algorithmique et efficacité à grande échelle. Une amélioration supplémentaire des performances nécessiterait soit une réduction du volume de communications globales, par exemple en modifiant l'algorithme pour limiter les synchronisations, soit une exécution sur plusieurs nœuds physiques afin de bénéficier d'une bande passante mémoire accrue. Ce travail illustre ainsi de manière concrète l'importance conjointe des choix algorithmiques et architecturaux dans la conception de solveurs parallèles performants.

A Annexe : Codes sources

Les codes sources complets utilisés pour l'implémentation du gradient conjugué parallèle, l'optimisation mémoire ainsi que l'analyse des performances sont fournis ci-dessous à titre de référence. Ces annexes ne sont pas prises en compte dans la pagination du rapport principal.

A.1 Implémentation du gradient conjugué parallèle

```
#!/usr/bin/env python
# -*- coding: utf-8 -*-


__author__ = 'F. Magoules'
__date__ = '2025-01-01'
__version__ = 'question (student)'
__example__ = 'mpiexec -n 4 python exercice5_conjugate_gradient_parallel_question.py'  # on a PC
    with windows 11


import os
import sys
numb_numpy_threads = 1  # setting the number of threads run by numpy
os.environ['OPENBLAS_NUM_THREADS'] = str(numb_numpy_threads)  # must be done before to importing
    numpy
import numpy # import numpy after setting the number of threads
import time
import exercice5_package

# Python MPI packages
import mpi4py
from mpi4py import MPI


def solve_conjugate_gradient_parallel(A, b, n, m, iteration_thresh, res_thresh):
    """
    Conjugate Gradient (CG).

    :references:
    Hestenes and Stiefel, 1952
    Magoules and Roux, 2013

    Parameters
    ----------
    A : numpy.ndarray
        -- dense matrix
    b : numpy.ndarray
        -- vector
    n : int
        -- number of rows of the matrix
    m : int
        -- number of columns of the matrix
    iteration_thresh : float
        -- maximum number of iterations
    res_thresh : float
        -- stopping ctiteria

    Returns
    -------
    x : numpy.ndarray
        -- solution (assembled) of the linear system
    iteration : int
        -- number of iteration

    """

    if MPI.COMM_WORLD.Get_rank() == 0:
        print('Iterations of parallel algorithm')
        sys.stdout.flush()

    # -- initialization of the conjugate gradient
    # initialize solution x
    x = numpy.zeros(n, dtype=A.dtype)

    # initialize gradient g
    g = -b.copy()

    # initialize vector w
```

```
    w = g.copy()

    # initialization of additional variables
    s1_assemb = numpy.empty(1, numpy.float64)
    s2_assemb = numpy.empty(1, numpy.float64)
    s3_assemb = numpy.empty(1, numpy.float64)
    s4_assemb = numpy.empty(1, numpy.float64)
    s5_assemb = numpy.empty(1, numpy.float64)
    w_assemb = numpy.empty(A.shape[1], numpy.float64)

    # scalar product: s = (g, g)
    s1 = numpy.vdot(g, g)
    # On met le scalaire dans un tableau pour le Allreduce
    s1_loc = numpy.array([s1], dtype=numpy.float64)
    MPI.COMM_WORLD.Allreduce(s1_loc, s1_assemb, op=MPI.SUM)

    # compute norm: sqrt( (g, g)
    res_init = numpy.sqrt(numpy.abs(s1_assemb[0]))
    res_norm = res_thresh
    vec_res_rel = numpy.zeros(iteration_thresh + 1, numpy.float64)

    # -- iteration of the conjugate gradient
    iteration = 0
    while res_norm >= res_thresh and iteration < iteration_thresh:
        # matrix-vector product: v = A * w
        # On rassemble tous les morceaux locaux de w dans w_assemb
        MPI.COMM_WORLD.Allgather(w, w_assemb)
        v = A.dot(w_assemb)

        # scalar product: s = (g, w)
        s1 = numpy.vdot(g, w)
        s1_loc = numpy.array([s1], dtype=numpy.float64)
        MPI.COMM_WORLD.Allreduce(s1_loc, s1_assemb, op=MPI.SUM)

        # scalar product: s = (v, w)
        s2 = numpy.vdot(v, w)
        s2_loc = numpy.array([s2], dtype=numpy.float64)
        MPI.COMM_WORLD.Allreduce(s2_loc, s2_assemb, op=MPI.SUM)

        # compute coefficient rho
        rho = - s1_assemb / s2_assemb

        # compute relative residual
        vec_res_rel[iteration] = numpy.linalg.norm(rho * w_assemb)

        # update x
        x = x + rho * w

        # update g
        g = g + rho * v

        # scalar product: s = (g, v)
        s3 = numpy.vdot(g, v)
        s3_loc = numpy.array([s3], dtype=numpy.float64)
        MPI.COMM_WORLD.Allreduce(s3_loc, s3_assemb, op=MPI.SUM)

        # scalar product: s = (v, w)
        s4 = numpy.vdot(v, w)
        s4_loc = numpy.array([s4], dtype=numpy.float64)
        MPI.COMM_WORLD.Allreduce(s4_loc, s4_assemb, op=MPI.SUM)

        # compute coefficient gamma
        gamma = - s3_assemb / s4_assemb

        # compute residual
        # On calcule (g, g) globalement pour avoir la norme
        g_norm2_loc = numpy.array([numpy.vdot(g, g)], dtype=numpy.float64)
        g_norm2_glob = numpy.empty(1, numpy.float64) # Variable temporaire
        MPI.COMM_WORLD.Allreduce(g_norm2_loc, g_norm2_glob, op=MPI.SUM)

        res_norm = numpy.sqrt(g_norm2_glob[0]) / res_init

        # update w
        w = g + gamma * w

        iteration = iteration + 1
        if MPI.COMM_WORLD.Get_rank() == 0:
            print('CG parallel iter {:d} : res {:.3e}'.format(iteration, res_norm))
            sys.stdout.flush()
```

```
147
148     # reconstruct global solution from local solution for automatic correction purpose
149     x_assemb = numpy.zeros(A.shape[1], numpy.float64)
150     MPI.COMM_WORLD.Allgather(x, x_assemb)
151
152     return x_assemb, x, iteration
153
154
155 def main():
156
157     MPI_COMM_WORLD = MPI.COMM_WORLD
158     size = MPI_COMM_WORLD.Get_size()
159     rank = MPI_COMM_WORLD.Get_rank()
160     hostname = MPI.Get_processor_name()
161
162     # initialization of the size of the global matrix
163     # MODIFY THE LINE BELOW TO CHANGE THE DIMENSION OF THE MATRIX
164     n = 2*3*4*5*6
165     n = exercice5_package.cmdLineParsing(n, rank, size)
166     # initialization of the maximum number of iterations
167     iteration_thresh = 100
168     # initialization of stopping criteria
169     res_thresh = 1.e-8
170
171     # initialize matrix and right hand side
172     # A, b = exercice5_package.assemble_block(n, size, rank)  <-- ANCIENNE LIGNE (   commenter ou
            supprimer)
173     A, b = exercice5_package.assemble_block_parallel(n, size, rank)
174
175     # solve with conjugate gradient
176     time_begin = time.time()
177     x_cgp_assemb, x_cgp_local, numb_iter = solve_conjugate_gradient_parallel(A, b, A.shape[0], A.
            shape[1], iteration_thresh, res_thresh)
178     time_end = time.time()
179     time_ellapsed = time_end - time_begin
180
181     # display solution and ellapsed time
182     if MPI.COMM_WORLD.Get_rank() == 0:
183         print('Value of iterative (parallel) solution =', x_cgp_assemb)
184         sys.stdout.flush()
185         print(f'Ellapsed time to compute iterative (parallel) solution = {time_ellapsed: .6f} (s)'
                )
186         print(f'(Parallel) computing speed = {n/time_ellapsed/1000: .2f} (Kvar/s)')
187         sys.stdout.flush()
188
189     # check accuracy of solution
190     if MPI.COMM_WORLD.Get_rank() == 0:
191         # bool, stg = exercice5_package.automatic_correction_is_solution_correct(n, x_cgp_assemb,
                epsilon=1.e-3)
192         # print('Automatic correction:', stg)
193         # sys.stdout.flush()
194         pass # Ajoute 'pass' pour  viter  une erreur d'indentation si tout est comment
195
196     return
197
198
199 if __name__ == "__main__":
200
201     main()
```

A.2 Bibliothèque d'assemblage distribuée

```python
#!/usr/bin/env python
# -*- coding: utf-8 -*-


__author__ = 'F. Magoules'
__date__ = '2025-01-01'
__version__ = 'automatic correction (students)'


import os
import numpy
import sys
import time
import argparse


def assemble_matrix(n):
    """

    Parameters
    ----------
    n : int
        -- number of rows of the matrix

    Returns
    -------
    mat : numpy.ndarray
        -- finite element matrix
    n : int
        -- number of rows of the matrix

    """
    mat = numpy.zeros((n, n), numpy.float64)
    for i in range(0, n):
        mat[i,i] = 4.
        if 0 < i:
            mat[i,i-1] = -1.
        if i < n-1:
            mat[i,i+1] = -1.
    mat[0,:] = 0.
    mat[:,0] = 0.
    mat[0,0] = 1.

    return mat, n


def automatic_correction_is_solution_correct(n, x_global_assemb, epsilon=1.e-3):
    """

    Parameters
    ----------
    n : int
        -- number of rows of the matrix
    x_global_assemb : numpy.ndarray
        -- global vector (assembled)
    epsilon : float
        -- numerical precision

    Returns
    -------
    bool : boolean
        -- True or False
    stg : string
        -- message

    """
    A, n = assemble_matrix(n)
    b = numpy.ones(n, dtype=numpy.float64)
    x_global = numpy.linalg.solve(A, b)
    if numpy.linalg.norm(abs(x_global_assemb - x_global)) < epsilon:
        bool = True
        stg = 'Congratulation, you pass.'
    else:
        bool = False
        stg = 'It seems that something is not correct... Please check your code.'

    return bool, stg
```

```
78
79
80  def assemble_block(dim, nbBlocks, indBlock):
81      """
82
83      Parameters
84      ----------
85      dim : int
86          -- number of rows of the global matrix
87      nbBlocks : int
88          -- number of row-blocks of the block-partition of the global matrix
89      indBlock : int
90          -- index of the current block
91
92      Returns
93      -------
94      A_local : numpy.ndarray
95          -- current row-block of the local matrix
96      b_local : numpy.ndarray
97          -- current row-block of the right hand side
98
99      """
100     A_global, dim = assemble_matrix(dim)
101
102     ibegin = int(indBlock * (A_global.shape[0] / nbBlocks))
103     iend = int((indBlock + 1) * (A_global.shape[0] / nbBlocks))
104     A_local = A_global[ibegin:iend, :]
105
106     b_global = numpy.ones(dim, dtype=numpy.float64)
107     b_local = b_global[ibegin:iend]
108
109     return A_local, b_local
110
111
112 def assemble_block_parallel(dim, nbBlocks, indBlock):
113     """
114     Parameters
115     ----------
116     dim : int
117         -- number of rows of the global matrix
118     nbBlocks : int
119         -- number of row-blocks (MPI processes)
120     indBlock : int
121         -- index of the current block (MPI rank)
122
123     Returns
124     -------
125     A_local : numpy.ndarray
126         -- current row-block of the local matrix
127     b_local : numpy.ndarray
128         -- current row-block of the right hand side
129     """
130     # 1. Calcul des bornes (indices globaux de d but et fin pour ce processus)
131     ibegin = int(indBlock * (dim / nbBlocks))
132     iend = int((indBlock + 1) * (dim / nbBlocks))
133
134     # 2. Allocation de la matrice LOCALE
135     # Elle a (iend - ibegin) lignes et 'dim' colonnes
136     rows_local = iend - ibegin
137     A_local = numpy.zeros((rows_local, dim), numpy.float64)
138
139     # 3. Remplissage intelligent (sans g n rer la matrice globale)
140     for i_loc in range(rows_local):
141         i_glob = ibegin + i_loc # L'indice r el de la ligne dans la matrice globale
142
143         # Condition limite stricte : Ligne 0 globale = [1, 0, 0, ...]
144         if i_glob == 0:
145             A_local[i_loc, 0] = 1.0
146         else:
147             # Diagonale Principale (valeur 4.0)
148             A_local[i_loc, i_glob] = 4.0
149
150             # Voisin Droit (valeur -1.0), sauf si on est au tout dernier bord
151             if i_glob < dim - 1:
152                 A_local[i_loc, i_glob + 1] = -1.0
153
154             # Voisin Gauche (valeur -1.0)
155             # ATTENTION : La colonne 0 est forc e    0 dans ce TP.
156             # Donc on ne met un -1    gauche QUE si (i_glob - 1) > 0.
```

```
157             if i_glob - 1 > 0:
158                 A_local[i_loc, i_glob - 1] = -1.0
159 
160     # 4. Allocation et initialisation du vecteur b local
161     b_local = numpy.ones(rows_local, numpy.float64)
162 
163     return A_local, b_local
164 
165 
166 def cmdLineParsing(DefaultSize, rank, numb_proc):
167     """
168     Command line parsing.
169     """
170 
171     # parser definition
172     parser = argparse.ArgumentParser()
173     parser.add_argument("--s", help="vector size (nb of variables)", default=DefaultSize, type=int
        )
174     # DefaultNt   = 1
175     # parser.add_argument("--nt", help="nb of threads run by Numpy operations in each process (>0)
        ", default=DefaultNt, type=int)
176 
177     # command line parsing
178     args = parser.parse_args()
179 
180     # checking of the command line arguments
181     if args.s <= 0:
182         if rank == 0:
183             sys.exit("Error: vector size must be an integer greater than 0!")
184         else:
185             sys.exit("")
186 
187     if args.s % numb_proc != 0:
188         if rank == 0:
189             sys.exit("Error: vector size must be a multiple of the number of processes!")
190         else:
191             sys.exit("")
192 
193     # if args.nt <= 0:
194     #     if Me == 0:
195     #         sys.exit("Error: number of threads per process must be greater than 0!")
196     #     else:
197     #         sys.exit("")
198 
199     return args.s
```

A.3 Script d'analyse et de visualisation des performances

```
import matplotlib.pyplot as plt
import numpy as np

# --- 1. DONN ES MESUR ES ---
procs = np.array([4, 8, 16, 32])
N = np.array([4000, 8000, 16000, 32000])
times = np.array([0.052606, 0.085875, 0.181634, 0.504138])

# --- 2. CALCULS DES M TRIQUES AVANC ES ---

# A. Vitesse de calcul (Kvar/s)
# Formule tir e de ton code MPI : n / time / 1000
speed_kvars = N / times / 1000

# B. Efficacit  du Weak Scaling (%)
# Id alement , le temps reste constant ( gal  au temps pour 4 procs).
# Efficacit  = Temps_Id al / Temps_R el
ideal_time = times[0]
efficiency = (ideal_time / times) * 100

# --- 3. CR ATION DES GRAPHIQUES ---
fig, (ax1, ax2, ax3) = plt.subplots(1, 3, figsize=(18, 6))
fig.suptitle('Analyse de Performance Avanc e - Gradient Conjugu  Parall le\n(Weak Scaling : N/P
    = 1000)', fontsize=16)

# GRAPHIQUE 1 : Temps d'ex cution (Le classique)
ax1.plot(procs, times, 'o-', color='b', linewidth=2, label='Mesures r elles')
ax1.axhline(y=ideal_time, color='r', linestyle='--', label='Id al (Temps constant)')
ax1.set_title("1. Temps d'ex cution")
ax1.set_xlabel('Nombre de Processus (P)')
ax1.set_ylabel('Temps (s)')
ax1.set_xticks(procs)
ax1.grid(True, linestyle=':', alpha=0.6)
ax1.legend()

# GRAPHIQUE 2 : Efficacit  (La "qualit " du parall lisme)
ax2.plot(procs, efficiency, 's-', color='green', linewidth=2, label='Efficacit ')
ax2.plot(procs, [100]*len(procs), 'r--', label='Id al (100%)')
ax2.set_title("2. Efficacit  du Weak Scaling")
ax2.set_xlabel('Nombre de Processus (P)')
ax2.set_ylabel('Efficacit  (%)')
ax2.set_ylim(0, 110) # Pour bien voir la chute
ax2.set_xticks(procs)
ax2.grid(True, linestyle=':', alpha=0.6)

# Annotation des valeurs
for p, eff in zip(procs, efficiency):
    ax2.annotate(f'{eff:.1f}%', (p, eff), textcoords="offset points", xytext=(0,10), ha='center')
ax2.legend()

# GRAPHIQUE 3 : D bit Agr g  (La puissance syst me)
# Id alement , la puissance totale devrait monter lin airement (x2 procs = x2 puissance)
ideal_speed = speed_kvars[0] * (procs / procs[0])
ax3.plot(procs, speed_kvars, 'd-', color='purple', linewidth=2, label='Vitesse R elle (Kvar/s)')
ax3.plot(procs, ideal_speed, 'r--', label='Id al (Scalabilit  lin aire)')
ax3.set_title("3. Puissance Agr g e (Throughput)")
ax3.set_xlabel('Nombre de Processus (P)')
ax3.set_ylabel('Vitesse Totale (Kvar/s)')
ax3.set_xticks(procs)
ax3.grid(True, linestyle=':', alpha=0.6)
ax3.legend()

plt.tight_layout()

# Sauvegarde
filename = 'analyse_complete_weak_scaling.png'
plt.savefig(filename)
print(f"Graphique g n r  : {filename}")
# plt.show()
```